

- Group number: 65
- Module: 2
- Emrik Dunvald 020208-5759, ADS
- Elias Samantzis 000715-6631, ADS
- Gusamanel@student.gu.se
- Gusdunvem@student.gu.se
- We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.
- Elias hours spent: 20 hours
- Emrik hours spent: 20 hours

The first paper called “The Netflix Prize” discusses the overall goal of the competition that Netflix created where they aimed to improve their recommendation algorithm. They describe their own algorithm which works on user ratings and computes the recommendation for a user based on what other similar users have rated highly, using multivariate regression. Their system is called cinematch and their goal was to improve this system by 10% by giving out a prize to any team which could do it. The second paper called “Lessons from the Netflix prize challenge” discusses the key strategies of the first winning team of the competition. One big takeaway from this paper was that it is extremely valuable to utilize models that incorporate more information than simple ratings. These highlights could be observed when the authors concluded that an ensemble of multiple specialized predictors was the most successful model for improving accuracy. The paper mentions the central and important role of regularization to avoid overfitting and make sure the model's predictions actually work in real-life scenarios. The authors pointed out that the most effective way of collecting data is to combine the implicit and explicit methods, to enrich the understanding of a user's preferences and to back each other up.

Pick two design features (e.g., regarding data, models, algorithms) that characterise the task and the solution in the papers and discuss how they may differ in another application of recommendation systems:

There are two different data collection methods that have been discussed in the papers: explicit and implicit data collection. In Netflix’s case, they rely on explicit data collection, i.e., the ratings the users give movies. In the paper ‘Lessons from the Netflix Prize Challenge’, the authors mention how the dataset provided by Netflix was built using this explicit data. The advantages of utilizing this type of data are that when users are willing to provide data, i.e., give ratings, the system works well with this type of data and methods such as collaborative filtering and ensemble methods. However, the system suffers when there is data sparsity (users don't rate

movies), as without any ratings, the recommender system can't recommend movies. On top of that, the explicit data might not work for recommender systems such as Spotify or TikTok, where clicks and views are more important than ratings or other types of direct user feedback.

This is where implicit data comes into play. Implicit data is collected through clicks, watch time, skips, purchases, views, or likes. This type of data collection is extremely useful for recommender applications such as Spotify, TikTok, Amazon, or even LinkedIn, where direct feedback from a user is perhaps not the most important aspect. Implicit data could even function well for a system such as Netflix's. For example, when ratings are nonexistent, implementing implicit data (the number of clicks a movie has) can act as a recommender instead of the ratings. Although this type of data is indirect and not a concrete expression of user preferences, they can act as indicators to guide the system toward making a well-founded recommendation for the user. However, the implicit data can also be very noisy, containing noise that dilutes the user's true preferences. For example, when users are browsing for a new movie, song, or product, they might explore new or different things from what they usually do, and this 'noise' can distract the recommender system from the user's true preferences after they are done exploring.

The second design feature we wanted to discuss is the prediction model which Netflix had designed before the competition. The cinematch system was a user rating based system which used Pearson correlation between users based on their ratings. After that, the model uses multivariate regression in order to compute the rating which a user would give a movie which they haven't seen before. The model is then evaluated on the root mean squared difference between the predicted rating and the actual rating. A similar system could probably be implemented for any other data parameter which you could correlate between users. If we imagine youtube as an example then it would be easy to see how they could compute a correlation between users based on their watch time of different videos. The algorithm could then compute what videos a user would be likely to watch. One problem we might encounter with this system however is with the size of the user base of youtube. Netflix, at the time of the competition, had a much smaller number of users compared to youtube which could cause problems if you want to compute the correlation between each user.

Code implementation description:

In the system we designed, we first begin by extracting all the columns that begin with the word 'genre_' in order to extract all the types of movie genres from the movie data set. We also collect user ratings and remove all zero ratings as they are unwanted because they cause null values when computing the user profile. Basically, we pre-process the data and remove all unwanted attributes while structuring the data so we can work with it.

The second step involves creating a user profile based on their ratings of the movies. Here, a user profile vector is created based on the weighted genre preferences, we take ratings from the

individual user and compute the weighted average. We compute this step because we want to aggregate the users past ratings and map them to genres, as this will allow us to compare the users preferences with other movies.

Lastly, we compute the cosine similarity between the user profile and the available movies. Since the user's profile is a weighted genre vector and each movie is represented as a genre vector, cosine similarity measures how close two vectors are in terms of directions, so in our case, it would tell us how well a movie aligns with the users preferences based on the angle between the vectors. The higher the cosine similarity score, the better fit for the user.

All these steps lead to an output of 5 recommended movies to the specific users. By implementing these steps, our system uses content-based filtering because it focuses on what each user personally likes rather than relying on other users' preferences. Instead of comparing users to find similarities (like collaborative filtering does), we look at the individual's past ratings and recommend movies that match their individual tastes. This is especially useful when there is not a lot of user data available since we do not need a big dataset of other users to make good recommendations. By using cosine similarity, we can measure how well a movie's genres align with the user's preferences and thus ensure that the recommendations are personalized and meaningful.

A strength of implementing a content-based filtering recommender system is that each recommendation is specifically tailored to suit individual user preferences. This also implies that there is no dependency on other users, meaning that this implementation can function with only one user.

A disadvantage of this implementation is that if new users are introduced with few movie ratings, then the system may not be able to recommend meaningful movie suggestions, as the user lacks ratings to base their preferences. The module's effectiveness is heavily dependent on having well-structured data, as the lack of structure and comprehensiveness can lead to miss-recommendations in the final output.

Code:

```
—  
import pandas as pd  
from sklearn.metrics.pairwise import cosine_similarity  
import numpy as np  
  
movies_df = pd.read_csv("movie_genres.csv")
```

```

ratings_df = pd.read_csv("user_reviews.csv")

print(movies_df.shape)
—

—

# Extracting all the columns from the movie data frame where the name genre_ is included
genre_columns = [col for col in movies_df.columns if 'genre_' in col]

# Setting the index of these features to be the movies title
movie_features = movies_df.set_index("movie_title")[genre_columns]

# Converting from wide to a long format where User stays as id
user_ratings = pd.melt(ratings_df, id_vars="User", var_name='movie_title',
value_name='Ratings')

# Removing all reviews with a rating of 0
reviews = user_ratings[user_ratings['Ratings'] > 0]

# Remove rows with 'Unnamed: 0' in the movie_title column because it was interfering with the:
Compute weighted average of genre features using ratings
reviews = reviews[reviews["movie_title"] != "Unnamed: 0"]

reviews
—

—

# To find the movie recommendation for a specific user by collecting the users preferences
def get_user_profile(user_id):

    # Getting the specific user ratings
    user Rated movies = reviews[reviews['User'] == user_id]

    # Debugging for error
    print("User Rated Movies:\n", user Rated movies.head())

    # Merging the two dataframes
    user_movies_selection = pd.merge(user Rated movies, movie_features, on='movie_title',
how='left')

```

```

    # Compute weighted average of genre features using ratings
    user_profile = np.dot(user_movies_selection[genre_columns].T,
user Rated_movies["Ratings"])

    # Normalize profile to prevent bias towards high ratings
    user_profile /= user_profile.sum()

    return user_profile

# Vincent, Edgar, Addilyn, Marlee and Javier
Vincent = get_user_profile("Vincent")
Edgar = get_user_profile("Edgar")
Addilyn = get_user_profile("Addilyn")
Marlee = get_user_profile("Marlee")
Javier = get_user_profile("Javier")
—
—

def generate_recommendations(user_profile, movie_features, num_recommendations=5):
    # Ensure user_profile is a 2D array
    user_profile = user_profile.reshape(1, -1)

    # Replace nan values in user profile with zeros
    user_profile = np.nan_to_num(user_profile)

    # Ensure movie_features is a 2D NumPy array
    movie_features_array = movie_features.fillna(0).values

    # Compute cosine similarity
    similarity_movie = cosine_similarity(user_profile, movie_features_array)

    # Convert similarity scores to DataFrame
    movie_recommendations = pd.DataFrame({ "movie_title": movie_features.index, "similarity":
similarity_movie.flatten()})

    # Recommend Top N Movies
    top_recommendations = movie_recommendations.sort_values(by="similarity",
ascending=False).head(num_recommendations)

    return top_recommendations

```

```
# Vincent, Edgar, Addilyn, Marlee and Javier
print(generate_recommendations(Vincent, movie_features))
print(generate_recommendations(Edgar, movie_features))
print(generate_recommendations(Addilyn, movie_features))
print(generate_recommendations(Marlee, movie_features))
print(generate_recommendations(Javier, movie_features))
```

Discussion

One issue of evaluating a recommendation system is the fact that the system can't be properly tested before being deployed. Since we are recommending items to users, we need the users to give us feedback on the recommendation in order to evaluate the system. It could maybe be possible to test the system before deployment by having a number of testers which you recommend products to, although the problem with this approach is that it won't be very representative of the real process since it is in a much more controlled environment.

It's difficult to know before implementing a recommendation system what type of data you are interested in. In some cases you might want explicit data while in other implicit data would benefit your recommender system more. However, this might be a fact you can only evaluate after implementing the recommender system. It is also difficult to choose the correct implicit parameter since there are many different aspects of the system which you could use to compute your recommendations. Choosing whether watch time or likes is the more relevant can be difficult and if you choose to incorporate both of them in the model then we have to be careful to not make the model too complex.